# Parallel Sorting Algorithms On Hybrid Heterogeneous Servers

## Contents

# Abstract

Sorting is one of the fundamental algorithms in computer science that is used in almost any area of computing with a large amount of data points or elements. The importance of efficient sorting algorithms is becoming increasingly important as the size and scale of data continues to grow.

Modern servers are increasingly heterogeneous - featuring multicore CPUs alongside GPU accelerators, each with distinct memory hierarchies and performance characteristics. Writing hybrid programs that actively distribute work across all of these processors, rather than relying on a single device, is key to making full use of such systems. This paper addresses the problem of how to optimally partition a sorting workload across the CPU and GPU components of such a server.

Because of their importance, efficient sorting algorithms have been the subject of much computer science research and many of the large CPU and GPU manufacturers such as Intel and NVIDIA have released mature and highly optimized sorting libraries such as NVIDIA's Thrust and Intel OneAPI. The goal of this paper is to make efficient use of these libraries in a hybrid heterogeneous system.

Sorting is one of the most fundamental algorithms in computer science that is used in a wide range of applications across data processing, databases and scientific computing. As datasets continue to grow in scale, efficient sorting becomes increasingly critical - and increasingly difficult, as modern high performance servers are no longer homogeneous but hybrid systems featuring multiccore CPUs alongside one or more GPU accelerators.

This project addresses the problem of how to optimally partition a sorting workload across the CPU and GPU components of such a server, specifically a system featuring one Intel Icelake CPU and two NVIDIA A40 GPUs. We estalbihs that radix sort is the most appropriate algorithm for this setting due to its input-order independence and near-linear time complexity, and we study the performance of two parallel implementations: NVIDIA's Thrust's LSD radix sort on the GPU and the Raduls MSD radix sort on the CPU.

A key finding is that these two implementations respond to increasing value range in opposite directions, meaning that processor speed cannot be described by a single function or array length alone - the correct speed function depends on the value range of the input. Applying the functional performance model of Lastovetsky and Reddy to our measured speed functions, we demonstrate that a workload balance of up to 94% is achievable across all three processors, compared to just $28 - 35\%$ for a naive equal partition. This confirms that the functional performance model, originally designed for matrix multiplication, can successfully be extended to sorting on heterogeneous hardware.


# 1 Introduction

As datasets continue to grow in scale, the performance of sorting algorithms becomes increasingly important across nearly every area of computing. At the same time, the hardware landscape has shifted dramatically: modern high-performance servers are no longer homogeneous single-processor systems, but hybrid platforms featuring multicore

CPUs alongside one or more GPU accelerators, each with distinct memory hierarchies, parallelism models, and peak throughputs.

Making full use of such a system is a non-trivial problem. A naive approach might run sorting entirely on the GPU, leaving the CPU idle, or divide the input array equally between all processors regardless of their different processing speeds. Both strategies leave performance on the table — the first by ignoring available compute, the second by making the slowest processor a bottleneck. The fundamental challenge is: given an input array of n elements and a set of p heterogeneous processors, how do we partition the work so that all processors finish at approximately the same time?

This paper investigates that problem in the specific context of sorting. We make the following contributions. First, we motivate and justify the choice of radix sort as the sorting algorithm for a heterogeneous setting, demonstrating that its runtime is independent of input ordering — a property that proves essential for a stable partitioning strategy. Second, we profile two parallel radix sort implementations across a wide range of input sizes and value ranges, revealing that their throughput varies along two distinct and interacting dimensions. Third, we apply the functional performance model of Lastovetsky and Reddy to derive optimal workload partitions for our system, and validate these against a naive equal-partition baseline. Finally, we discuss the practical considerations of operating on a shared server and the implications for future work toward a production-ready heterogeneous sorting system.

# 2 Related Work

- *Programming Massively Parallel Processors, Version 4* — David B. Kirk, Wen-mei W. Hwu

  The theoretical foundation of the GPU radix sort implementation used in this paper is drawn from Programming Massively Parallel Processors by Kirk and Hwu. This book also helped me understand the inner workings of GPU parallel programming, the main focus of this book is CUDA programming which only ended up being used sparingly in the final project but was important in developing my understanding of parallel programming. The implementation and analysis in Section 6 of this paper builds directly on this material, extending it to measure practical performance characteristics across varying input sizes and values ranges on modern NVIDIA hardware.

- *High Performance Heterogeneous Computing* — Alexey L. Lastovetsky and Jack J. Dongarra

  The broader context for heterogeneous computing is established by Lastovetsky and Dongarra in High Performing Computing. Their work addresses the fundamental challenge of efficiently utilizing systems where processors have different architecture and performance characteristics.

  The data partitioning algorithm applied in Section 7 is taken directly from Lastovetsky and Reddy's 2007 paper Data Partitioning with a Functional Performance Model of Heterogeneous Processors. Their key contribution is the functional performance model, which treats processor speed as a continuous function of problem size rather than a fixed constant.

- *O*penH: A Novel Programming Model and API for Developing Portable Parallel Programs on Heterogeneous Hybrid Servers [2]

  This paper was my introduction to the concept of developing programs for Heterogeneous Hybrid Servers, the server this project was built for is the same as the server used in this paper, many concepts in their paper were introduced for me through this paper.

# 3 Architecture Specifications

## 3.1 Heterogeneous Systems

A heterogeneous server is a server featuring multicore CPU processors hosting multicore accelerators. A hybrid parallel program signifies a program that divides the workload heterogeneously between the multicore CPU processors and accelerators of a server to provide maximum resource utilization and performance on the server.

## 3.2 Nvidia GPUs

Our system contains two NVIDIA A40 GPUs. NVIDIA A40 GPUs are equipped with 48 GiB of GDDR6 memory access, a 384-bit bus, delivering up to 696 GiB/s peak memory bandwidth. Via NVLink two A40s can be bridged together to expand usable memory to 96 GiB, making it well suited for large datasets and complex models. [1]

## 3.3 Intel CPUs

Our system contains one Intel Icelake multicore CPU - their processors are manufactured on Intel's 10nm process and feature two AVX-512 units per core, enabling up to 16 double precision FLOPS per cycle.

# 4 Tools For Parallel Programming

## 4.1 OpenMP

OpenMP is a parallel programming model for shared memory and distributed shared memory multiprocessors. It is comprised of a set of compiler directives that describe the parallelisms in the source code, along with a supporting library of subroutines available to the application.

The directives are instructional notes to any compiler supporting OpenMP. They take the form of source code comments (in Fortran) or `#pragma` (in C/C++) in order to enhance applicationportability when porting to non-OpenMP environments. [3]

## 4.2 OpenACC

In contrast to current mainstream GPU programming, such as CUDA and OpenCL, where more explicit compute and data management is necessary, porting of legacy CPU-based applications with OpenACC requires only code annotations without any significant

structural changes in the original code, which allows considerable simplification and productivity improvement when hybridizing existing applications.

Programming with OpenACC directives, while greatly simplified, is not as flexible as using CUDA or OpenCL. For example, both CUDA and OpenCL provide fine-grained synchronization primitives, such as thread synchronization and atomic operations, whereas OpenACC does not. [5]

## 4.3 CUDA

CUDA is a minimal extension of the C and C++ programming languages. A programmer writes a serial program that calls parallel kernels, which can be simple functions or full programs. The CUDA program executes parallel kernels across a set of parallel threads on the GPU. The programmer organizes these threads into a hierarchy of thread blocks and grids.

The CUDA programming model is similar in style to a single-program multiple data (SPMD) software model — it expresses parallelism explicitly, and each kernel executes on a fixed number of threads. However, CUDA is more flexible than most SPMD implementations because each kernel call dynamically creates a new grid with the right number of thread blocks and threads for the application step.

## 4.4 NVIDIA Thrust

Thrust is a C++ template library for CUDA based on the Standard Template Library (STL). Thrust allows you to implement high performance parallel applications with minimal programming effort through a high-level interface that is fully interoperable with CUDA C. Thrust includes support for algorithms such as sort, scan, transform and reduce, and these algorithms have been highly optimized and run more efficiently than a single developer could produce on their own in a reasonable amount of time.

## 4.5 Pthreads

Pthreads is a standardized model for dividing a program into substacks whose execution can be interleaved or run in parallel. The "P" in Pthreads come from POSIX (Portable Operating System Interface), the family of IEEE operating system interface standards in which Pthreads is defined. [4]

## 4.6 Intel TBB

Intel Threading Building Blocks is a C++ runtime library that abstracts the low-level threading details necessary for effectively utilizing multi-core processors. It uses C++ templates to eliminate the need to create and manage threads. The library consists of data structures and algorithms that simplify parallel programming in C++ by avoiding requiring a programmer to use native threading packages such as POSIX. [8]

## 4.7 Compilers

Due to many of the programming models for efficient parallel programming on CPUs and GPUs being vendor locked and managed by companies for specific use on their hardware,

in a heterogeneous system it can be necessary to use multiple compilers, compile files separately and then link them together.

### 4.7.1 NVCC

NVIDIA CUDA Compiler is a proprietary compiler by NVIDIA intended for use with CUDA. The compilation trajectory involves several splitting, compilation, preprocessing and merging steps. It is the purpose of NVCC to hide the intricate details of CUDA compilation from developers. All non-CUDA compilation steps are forwarded to a C++ host compiler that is supported by NVCC, and NVCC translates its options to appropriate host compiler command line options. [6]

### 4.7.2 ICPX

Intel C++ Compiler is a proprietary compiler by Intel intended for use across Intel processor architectures. The compilation process leverages advanced optimization techniques including auto-vectorization, interprocedural optimization, and profile-guided optimization steps. It is the purpose of the Intel C++ Compiler to maximize application performance on Intel hardware by exposing low-level architectural features to developers. Full compatibility with standard C++ host toolchains is maintained, and the compiler translates its optimization flags to appropriate platform-specific instructions targeting Intel's SSE, AVX and other SIMD instruction sets. [10]

### 4.7.3 GNU Compiler

GNU Compiler Collection is a free and open-source compiler by the GNU Project intended for use across a wide range of processor architecture and operating systems. The compilation process involves several stages including preprocessing, compilation, assembly and linking steps. It is the purpose of GCC to provide a portable, standards-compliment compiler that produces highly optimized native code across diverse hardware targets. Support for multiple programming languages is provided, including C, C++ and Fortran, making it well suited for heterogeneous codebases. GCC serves as the default host compiler for NVCC. [11]

# 5 Parallel Sorting Algorithms

## 5.1 Definition Of Sorting

Sorting is one of the earliest applications for computers. A sorting algorithm arranges the elements of a list into a certain order. The order to be enforced by a sorting algorithm depends on the nature of these elements. Any sorting algorithm must satisfy the following two conditions:

1. The output is in either nondecreasing or nonincreasing order. For nondecreasing order, each element is no smaller than the previous element according to the desired order. For nonincreasing order, each element is no larger than the previous element according to the desired order.
2. The output is a permutation of the input. That is, the algorithm must retain all of the original input elements while reordering them into the output. [7]

## 5.2 Types Of Sorting Algorithms

Sorting algorithms can be classified into stable and unstable algorithms. A stable sort algorithm preserves the original order of appearance when two elements have equal key value.

Sorting algorithms can also be classified into comparison-based and non-comparison-based algorithms. Comparison sorting algorithms arrange elements by comparing pairs and cannot achieve a time complexity greater than $O(n \log n)$. Non-comparison algorithms exploit numerical properties to achieve a faster and potentially linear $O(n)$ time.

Table 1: Classification of parallel sorting algorithms

| | **Comparison Based** | **Non-Comparison Based** |
|---|---|---|
| **Stable** | Parallel Merge Sort | LSD Radix Sort |
| **Unstable** | Parallel Quick Sort | MSD Radix Sort |

## 5.3 Time Complexity

For sorting of large datasets we want to pick algorithms with linear time complexity. Radix sort contains downsides for sorting datasets with large integer values, but parallel sorting on a heterogeneous system will usually be done on datasets where the size of the input makes time complexity more important.

Table 2: Time complexity of parallel sorting algorithms

| **Sorting Algorithm** | **Best Case** | **Average Case** | **Worst Case** |
|---|---|---|---|
| LSD Radix Sort | $O(n \cdot d)$ | $O(n \cdot d)$ | $O(n \cdot d)$ |
| MSD Radix Sort | $O(n)$ | $O(n \cdot d)$ | $O(n \cdot d)$ |
| Parallel Merge Sort | $O(n \cdot \log(n))$ | $O(n \cdot \log(n))$ | $O(n \cdot \log(n))$ |
| Parallel Quick Sort | $O(n \cdot \log(n))$ | $O(n \cdot \log(n))$ | $O(n^2)$ |

## 5.4 Testing On Sequential CPU

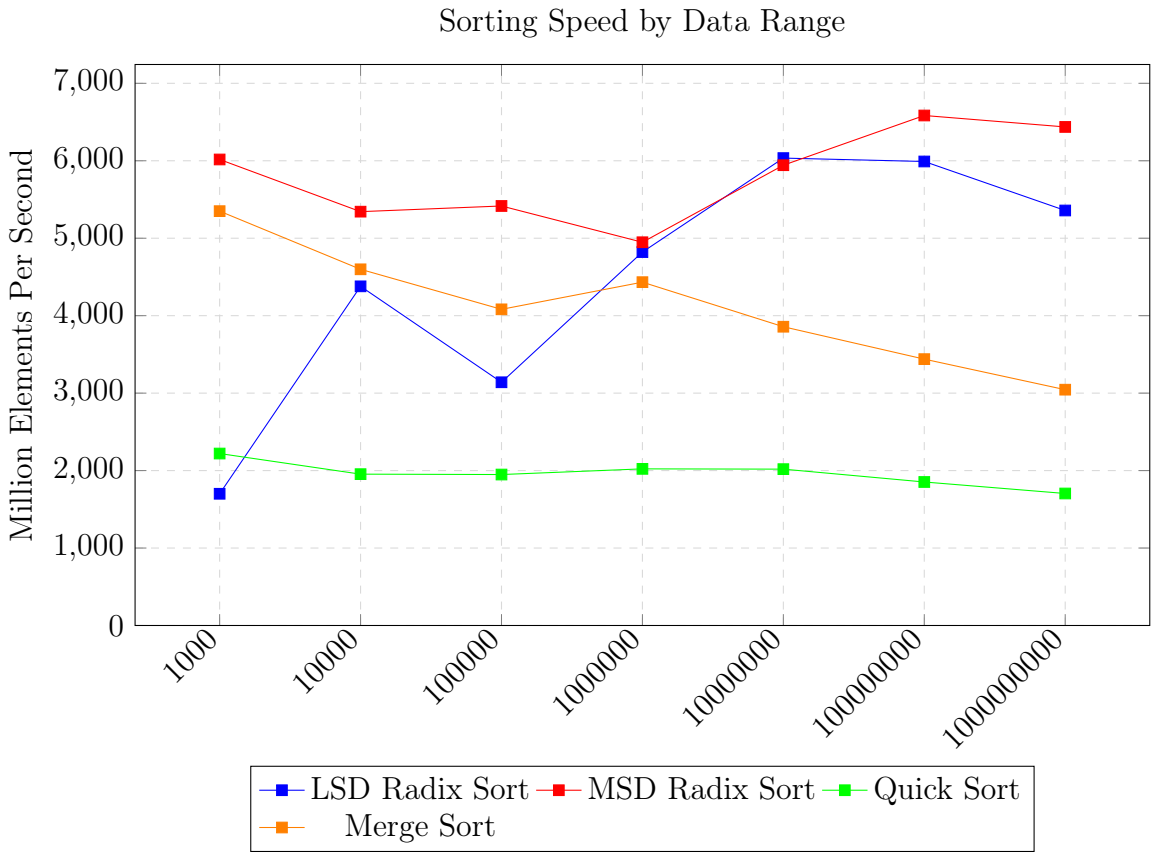


Figure 1: Elements Sorted Per Millisecond

Figure 1 presents the sorting throughput of four algorithms; LSD Radix Sort, MSD Radix Sort, Quick Sort and Merge Sort on a sequential single threaded CPU, measuring in million elements per second across input array lengths from $10^3$ to $10^8$ elements, with all values drawn from a fixed range of 0 to 100,000,000. All four algorithms exhibit increasing throughput as array size grows, reflecting improved amortization of fixed overheads at larger problem sizes. Both radix sort variants consistently outperform the comparison based algorithms across all tested array lengths, with LSD Radix Sort achieving the highest throughput overall. Quick Sort and Merge Sort trail behind, constrained by their O(n log n) complexity. These results confirm that for large input arrays on a sequential CPU, non-comparison-based sorting algorithms offer a significant performance advantage, motivating the adoption of radix sort as the algorithm of choice for the parallel and heterogeneous implementations explored in the following sections.

# 6 Understanding Parallel Radix Sort Algorithm

To further understand why these trend lines appear, we can analyze the Radix Sort Implementation in Chapter 13 of Programming Massively Parallel Processors.

```
1 __global__ void radix sort_iter(unsigned int* input, unsigned int*
    output, unsigned int* bits, unsigned int N, unsigned int iter){
2   unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    unsigned int key, bit;
    if(i < N){
        key = input[i];
        bit = (key >> iter) & 1;
        bits[i] = bit;
    }

    exclusiveScan(bits, N);
    if(i < N){
        unsigned int numOnesBefore = bits[i];
        unsigned int numOnesTotal = bits[i];
        unsigned int dst = (bit == 0) ? (i - numOnesBefore)
                                      : (N - numOnesTotal -
                                         numOnesBefore);
        output[dst] = key;
    }
}
```

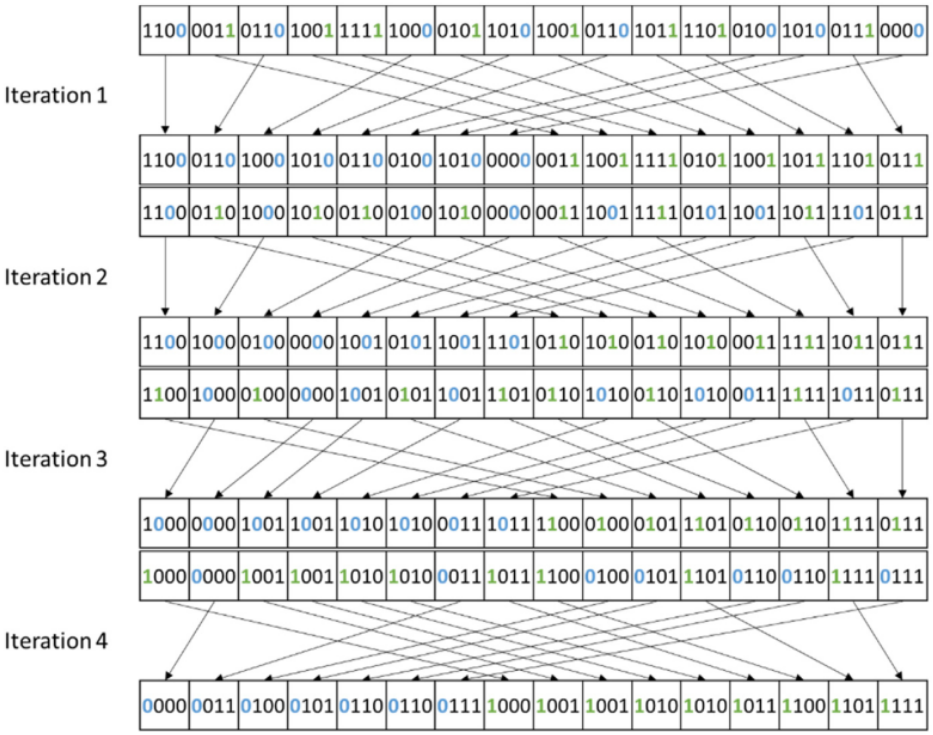


Figure 2: Visualization of Radix Sort

Quoted from Chapter 13 of Programming Massively Parallel Processors

Since radix sort processes one "digit" (a fixed-size slice of bits) per iteration, you need enough iterations to cover every bit in the key. With a 1-bit radix on 4-bit keys, for example you need exactly 4 iterations - one per bit.

Radix size is the main lever for reducing that count. By processing more bits per iteration you can cover the full key length in fewer passes. The generate rule is that for r-bit radix

applied to N-bit keys, you only need N/r iterations.

But we can't just keep increasing the radix to minimize iterations indefinitely, because a larger radix creates two new costs that push back. - each thread block ends up with more local buckets containing fewer keys each, which reduces the opportunity for coalesced memory writes - the table used for the global exclusive scan grows with the number of buckets (2r̂ rows), so its overhead scales up too

NVIDIA Thrust uses an 8-bit radix (ie radix 256) meaning that if our input array elements are in the range 2$\hat{}$(8*1)=256 we will have one iterations.

$2^{8*1}$ = 256 - 1 iteration

$2^{8*2}$ = 65536 - 2 iterations

$2^{8*3}$ = 16777216 - 3 iterations

$2^{8*4}$ = 4294967296 - 4 iterations

and so on..

## 6.1 Implementing Parallel Radix Sort

### 6.1.1 GPU - NVIDIA Thrust

Implementing radix sort on the GPU is quite straightforward because you can make use of NVIDIA Thrust which is a library created by NVIDIA and covers many efficiently parallelized algorithms such as sorting, transform and exclusive scan. If it possible to implement your own radix sort algorithm in CUDA following the guide in Programming Massively Parallel Processors shown above. However fully optimizing it can prove to be quite difficult, and making use of libraries when available is good practice.

### 6.1.2 CPU - Raduls

However implementing parallel LSD radix sort on the CPU proved to be much more complicated than I expected, mainly because there are no libraries which support it. All the CPU sorting libraries released by official vendors such as Intel use comparison based sorting algorithms such as Merge Sort and Quick Sort, these are okay but don't have the stability and reliability of radix sort.

The only radix sort implementation I found for the CPU was implementing a MSD variant of the radix sort algorithm, and as we'll see this led to some strange results.

## 6.2 Measuring Performance

### 6.2.1 Comparison To Typical Best and Worst Case

In most sorting algorithms the difference between the best case and the worst case scenario is defined by the ordering of the input array, a typical best case scenario would be one in which the original array is fully sorted, and a typical worst case scenario would be one in which the input array is fully reversed. For applications on a heterogeneous system this could lead to many problems because the difference in runtime between the CPU and the GPU could change along with the ordering of the inputted array.

Table 3: Comparison of typical "best case" and "worst case" in GPU and CPU sorting

| Input Range | Unordered GPU | Ordered GPU | Unordered CPU | Ordered CPU |
|---|---|---|---|---|
| 100000000 | 258 | 270 | 460 | 465 |
| 200000000 | 515 | 503 | 816 | 812 |
| 300000000 | 774 | 773 | 1145 | 1147 |
| 400000000 | 1028 | 1028 | 1487 | 1483 |
| 500000000 | 1289 | 1291 | 1833 | 1829 |

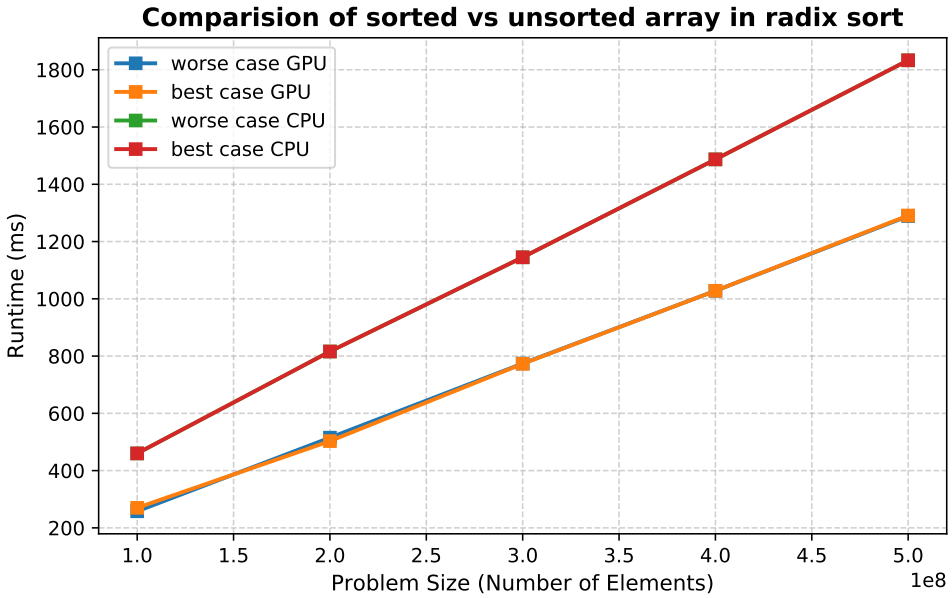


Figure 3: Typical Best and Worst Case

However, in radix sort the ordering of the input array has no effect on the runtime. Even a completely reversed array will have the same runtime through the radix sort array as an already fully sorted. This is a potential downside if we are dealing with inputs which are generally nearly sorted; however, for our use case on a hybrid heterogeneous server this is very beneficial because it means we can apply the same partitioning algorithm regardless of the input.

Additionally we see no difference in the pattern between MSD Radix Sort and LSD Radix Sort which is helpful for our implementation of a partitioning algorithm.

### 6.2.2 Comparison of Radix Sort Best and Worst Case

Table 4: Runtime Analysis of Functional Model in (ms) with Range

| Input range | GPU (ms) | CPU (ms) | ratio |
|---|---|---|---|
| $2^{8*1}$ | 234 | 447 | 0.52349 |
| $2^{8*2}$ | 246 | 425 | 0.578824 |
| $2^{8*3}$ | 254 | 414 | 0.613527 |
| $2^{8*4}$ | 273 | 377 | 0.724138 |
| $2^{8*5}$ | 282 | 321 | 0.878505 |
| $2^{8*6}$ | 293 | 273 | 1.07326 |
| $2^{8*7}$ | 288 | 222 | 1.2973 |
| $2^{8*8}$ | 312 | 169 | 1.84615 |

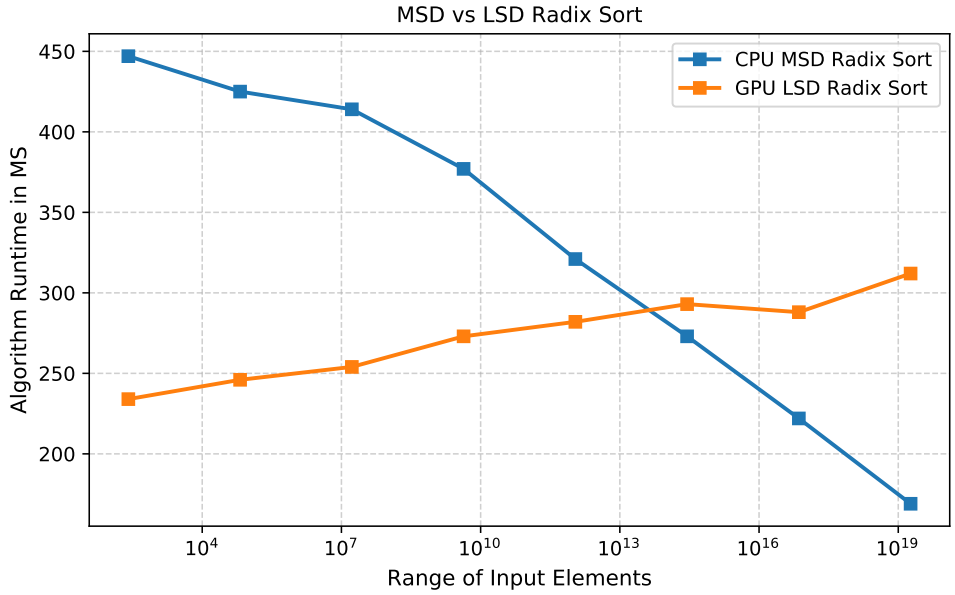


Figure 4: Differences Between MSD and LSD Radix Sort

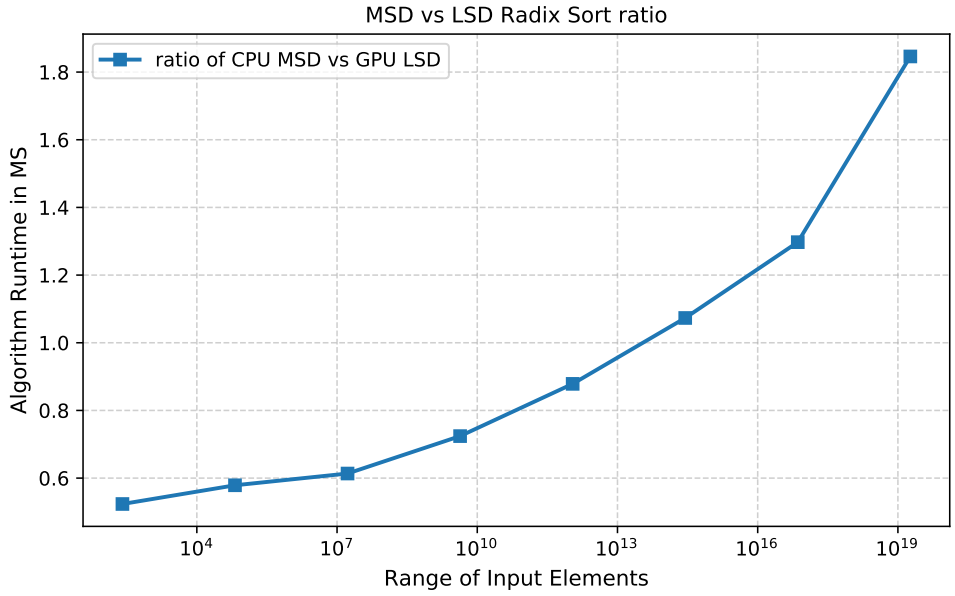


Figure 5: Input Array of length 100000000

Table 4 presents the runtime of GPU LSD Radix Sort and CPU MSD Radix Sort across input ranges from $2^8$ to $2^{64}$ with 100 million elements per run averaged over 10 iterations.

The results reveal that the two algorithms respond to increasing input range in opposite directions. As the range of input values widen from $2^8$ to $2^{64}$, GPU LSD Radix Sort decreases from 234ms to 312ms. This is expected behaviour, NVIDIA Thrust uses an 8-bit radix, meaning that each additional byte of value range introduces an additional pass over all 100 million elements. Wider ranges require more passes and, therefore, more time.

CPU MSD Radix Sort exhibits the opposite trend, improving from 447ms at $2^8$ down to 169ms at $2^{64}$. This is because MSD processes keys from the most significant byte first and recurses only as deeply as necessary per partition. For narrow ranges, nearly all elements share the same high order bytes, forcing the algorithm to recurse deeply through effectively empty upper-level buckets before reaching meaningful distributions. For wide ranges, values are immediately distributed across all 256 top level buckets, allowing most branches to terminate after very few recursive levels.

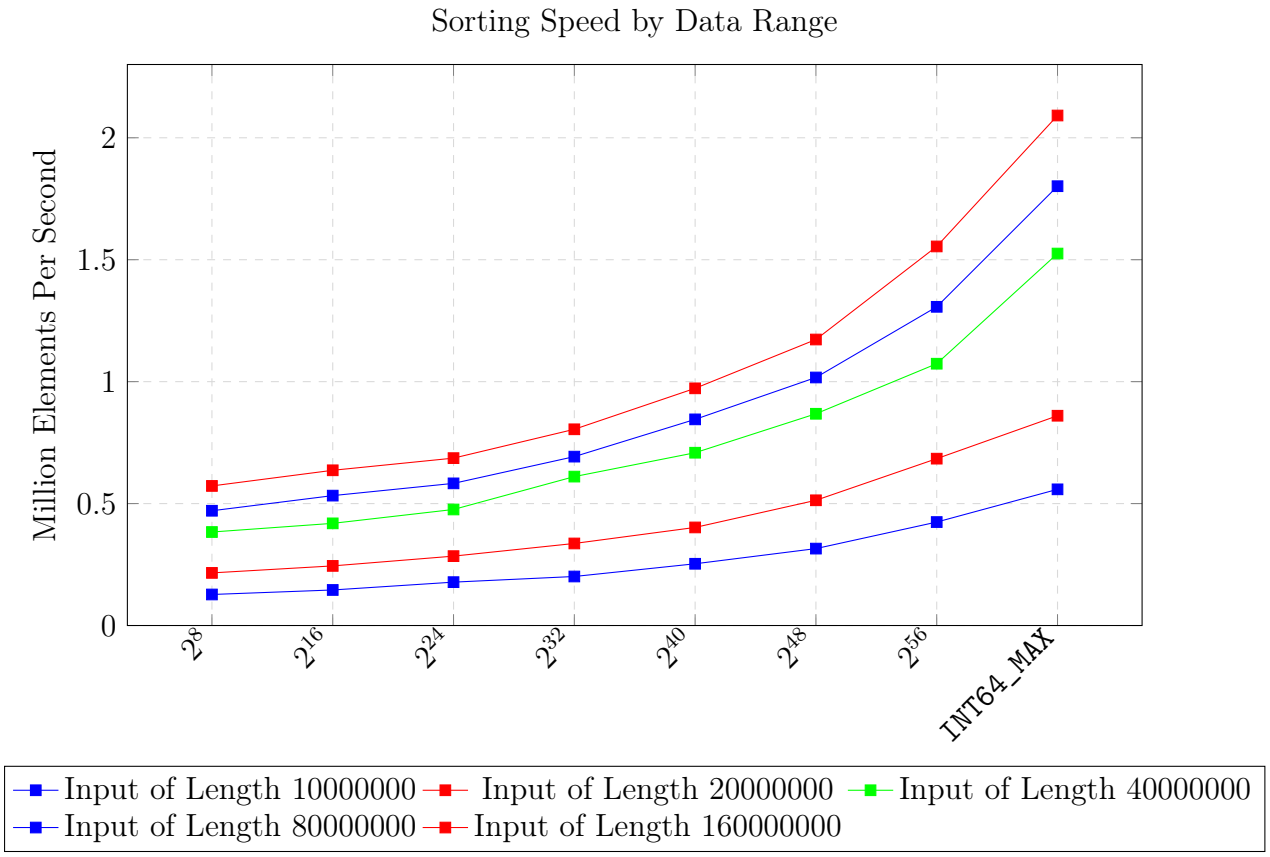


Figure 6: Ratio of CPU MSD to GPU LSD Radix Sort runtime across value ranges 2 to 2, for input lengths from 10 million to 160 million elements

Figure 6 extends the analysis of Figure 5 by plotting the CPU/GPU runtime ratio across multiple input lengths simultaneously, ranging from 10 million to 320 million elements. Two clear trends emerge. First, all lines share the same upward trajectory as value range widens, confirming the earlier finding that increasing range penalizes GPU LSD sort more than CPU MSD sort due to the additional passes required by the 8-bit radix. Second, larger arrays produce a consistently higher ratio across all ranges, meaning the CPU becomes relatively more competitive as array length grows.

Together these two dimensions, value range and array length interact to determine the optimal partition. Notably, this means a partitioning algorithm calibrated purely on array size without accounting for value range, will produce suboptimal allocations in workloads where the range of input values is known in advance.

For the purpose of this paper, we will fix the value range and treat array size as the sole input to the partitioning algorithm, which is sufficient for our experimental setup, but extending the model to incorporate range as a second dimension repreents a natural direction for future work.

## 6.3 Conclusion on Radix Sort

Given the facts discussed in section 6.1 and section 6.2 we can conclude that radix sort is the best algorithm for our needs.

# 7 Performance On A Heterogeneous System

An immediate implication from the heterogeneity of processors in a network of computers is that the processors run at different speeds. A good parallel application for a homogeneous distributed memory multiprocessor system tries to evenly distribute computations over available processors.

A good parallel application for the heterogeneous platform must distribute computations unevenly, taking into account the difference in processor speed. The faster the processor is, the more computations it must perform.

The speed is defined as the number of computation units performed by the processor per one time unit. The model is application specific. In particular, this means that the computation unit can be defined differently for different applications — the important requirement is that the computation unit does not have to vary during the execution of the application.

## 7.1 Data Partitioning

To make the best use of our sorting algorithms across multiple separate processors, we will divide our input data into separate categories each to be sorted on that processor and then merged back together on the CPU to form a fully sorted array.

However, we have a problem because unlike the Matrix Multiplication presented in Lastovetsky and Reddy 2007 our performance is effected by both array input length and the range of the elements in the array.

## 7.2 Measuring Million Elements Per Second with Input Array Size and Data Range

Here we can see that there are two dimensions to our variance in processing speed, one is the range of the input data. We can see this by looking at the sorting speed of input arrays with a constant length of 55,360,000, each varying in their element range from $0 - 2^{2*1}$ to $0 -$ int64_max.

Table 5: Million Elements Per Second at Array Length 655,360,000

| Data Range | GPU 1 Speed | GPU 2 Speed | CPU Speed | CPU to GPU ratio |
|---|---|---|---|---|
| $2^{8*1}$ | 499.72 | 507.94 | 261.12 | 0.522 |
| $2^{8*2}$ | 475.45 | 477.73 | 263.61 | 0.554 |
| $2^{8*3}$ | 430.06 | 439.04 | 264.56 | 0.615 |
| $2^{8*4}$ | 430.64 | 431.93 | 262.49 | 0.609 |
| $2^{8*5}$ | 412.39 | 418.08 | 295.72 | 0.717 |
| $2^{8*6}$ | 392.39 | 395.34 | 314.61 | 0.801 |
| $2^{8*7}$ | 385.42 | 384.82 | 364.24 | 0.945 |
| int64_max | 368.34 | 373.42 | 425.69 | 1.155 |

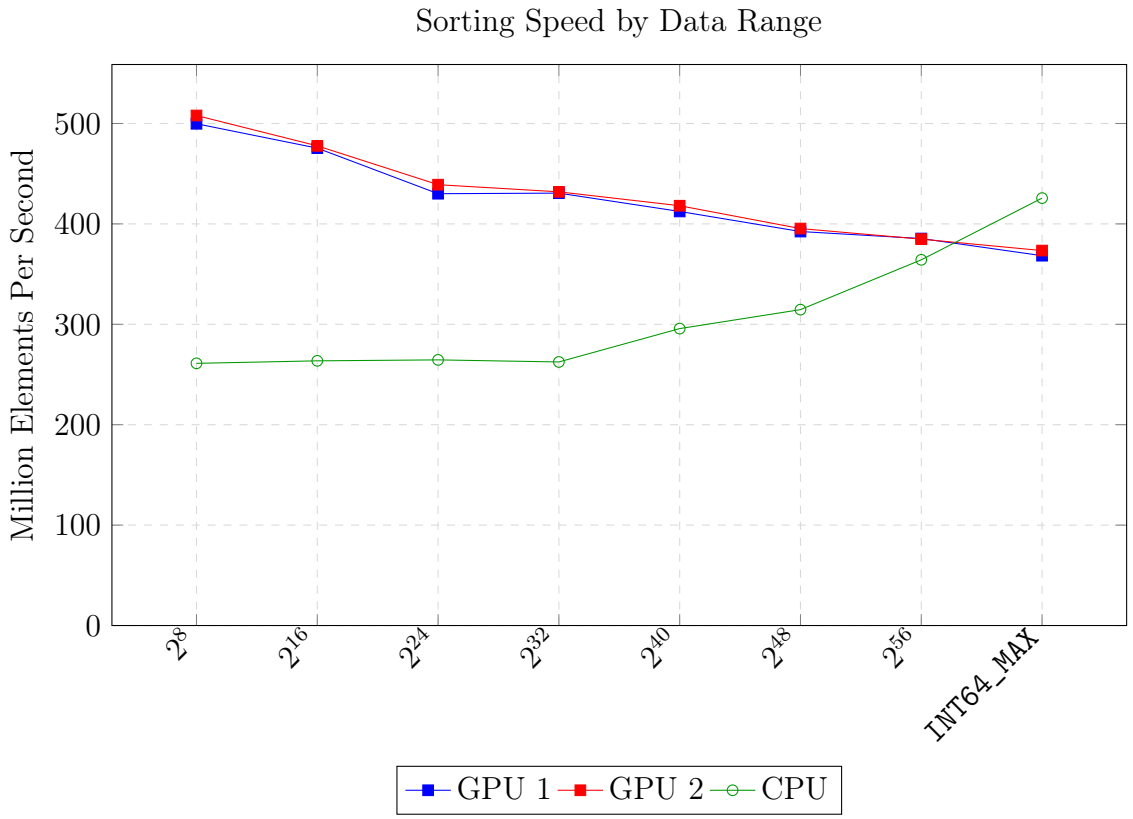


Figure 7: Million Elements Per Second at Array Length 655,360,000

### 7.2.1 Varying Length Array 1

The second variance comes from the length of an array where each element is drawn from a constant value range. Table 6 presents the sorting speed of GPU 1, GPU 2 and CPU RADULS across array lengths from 10,000 to 655,360,000 elements, with all values in the range $2 - 2^{56}$.

Table 6: Million Elements Per Second at range 0-$2^{(8*7)}$*withvaryingarraylength*

| Data Range | GPU 1 Speed | GPU 2 Speed | GPU1/GPU2 ratio | CPU Speed | CPU to GPU ratio |
|---|---|---|---|---|---|
| 10000 | 10.88 | 11.19 | 0.97 | 0.19 | 0.0179 |
| 20000 | 22.09 | 22.81 | 0.96 | 0.47 | 0.0212 |
| 40000 | 43.43 | 44.80 | 0.96 | 0.83 | 0.0192 |
| 80000 | 80.78 | 83.04 | 0.97 | 1.60 | 0.0198 |
| 160000 | 133.32 | 137.11 | 0.97 | 3.58 | 0.0269 |
| 320000 | 185.93 | 191.11 | 0.97 | 7.03 | 0.0378 |
| 640000 | 238.06 | 244.85 | 0.97 | 9.90 | 0.0415 |
| 1280000 | 276.89 | 281.52 | 0.98 | 19.01 | 0.0686 |
| 2560000 | 325.02 | 332.13 | 0.97 | 37.15 | 0.1143 |
| 5120000 | 397.75 | 403.60 | 0.98 | 64.06 | 0.1610 |
| 10240000 | 408.51 | 409.88 | 0.99 | 123.62 | 0.3026 |
| 20480000 | 385.34 | 385.60 | 0.99 | 184.70 | 0.4793 |
| 40960000 | 383.31 | 383.34 | 0.99 | 269.21 | 0.7023 |
| 81920000 | 384.72 | 383.54 | 1.00 | 326.95 | 0.8498 |
| 163840000 | 385.34 | 385.65 | 0.99 | 374.85 | 0.9727 |
| 327680000 | 385.94 | 385.11 | 1.00 | 366.11 | 0.9486 |
| 655360000 | 385.42 | 384.82 | 1.00 | 364.24 | 0.9450 |

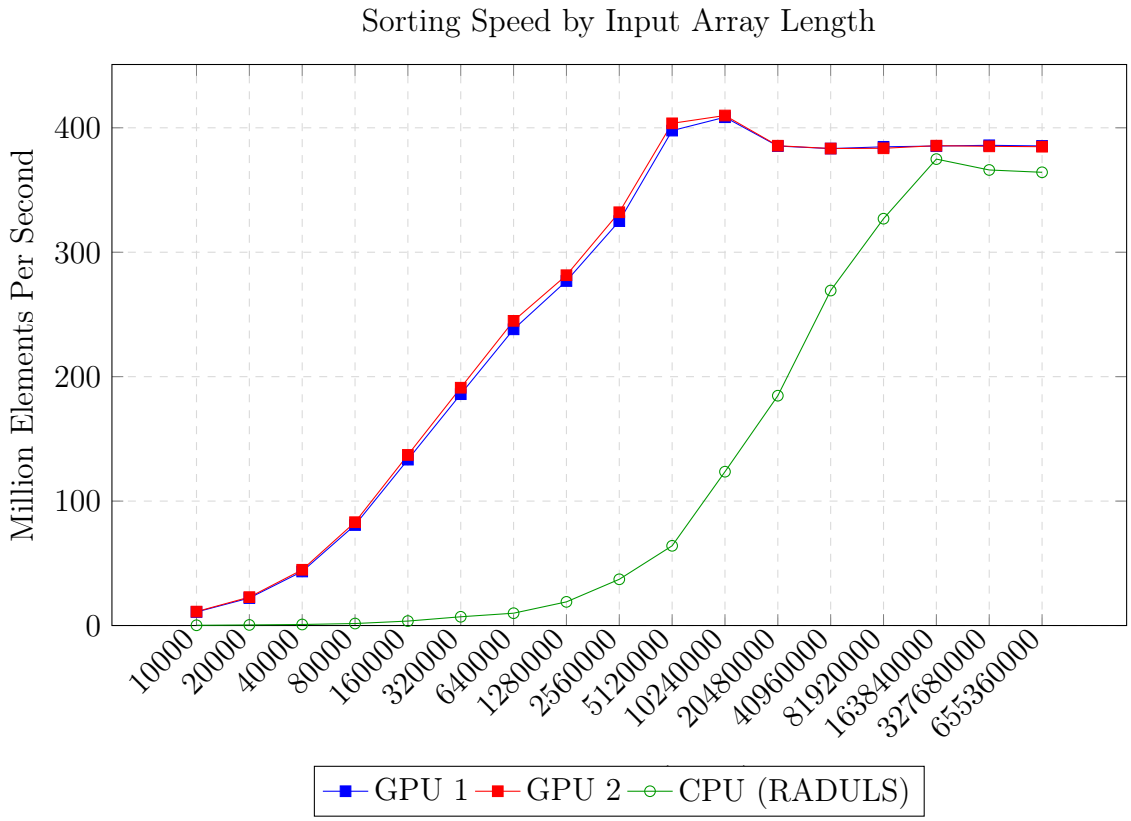


Figure 8: Sorting throughput of GPU 1, GPU 2 and CPU (RADULS) at value range 0 – $2^{56}$ with varying array length.

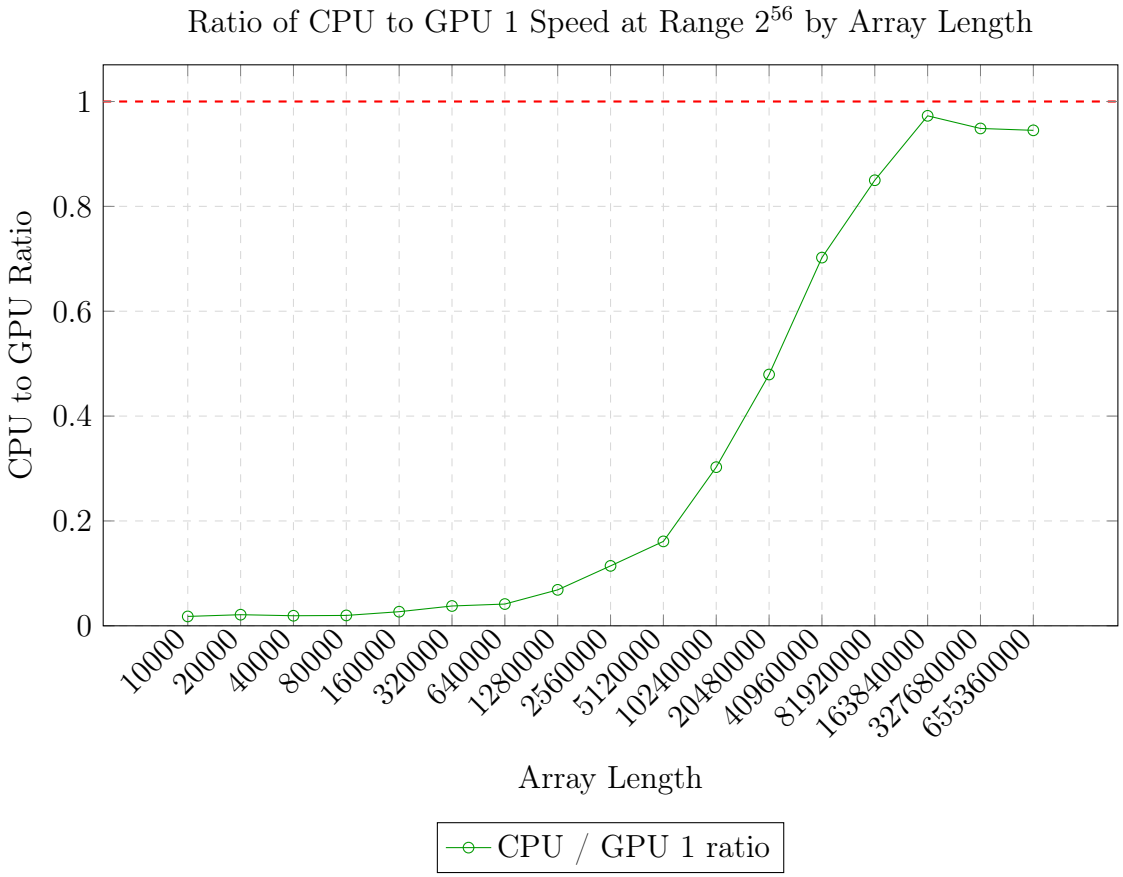


Figure 9: Ratio of CPU to GPU 1 speed at range 0–$2^{56}$ with varying array length. A ratio of 1.0 (red dashed line) indicates equal performance.

Both GPU curves follow the same pattern: throughput rises steeply from around 100 MELems/sec at 10,000 elements, saturating at approximately 385 MElems/sec once the array is large enough to fully occupy the GPU's parallel execution units, around 5-10 million elements. Beyond this point additional elements do not increase throughput, confirming that the GPU is memory bandwidth bound at this range.

The CPU curve tells a different story. At small sizes RADULS achieve under 1 MElems/sec reflecting the significant overhead of spawning hardware threads relative to the actual work performed. Throughput grows steadily as array size increases, reaching approximately 374 MElems/sec at 163 million elements - nearly matching GPU performance. The CPU to GPU ratio approaches but does not exceed 1.0 at this range, peaking at 0.97 before dropping slightly at larger sizes, suggesting the CPU is also approaching its memory bandwidth ceiling.

### 7.2.2 Varying Length Array 2

Table 7 repeats the same analysis with a narrower value range of $0 - 2^{16}$, allowing direct comparison of how value range affects the relative performance of each processor.

Table 7: Million Elements Per Second at range 0–$2^{16}$ with varying array length

| Array Length | GPU 1 Speed | GPU 2 Speed | GPU1/GPU2 | CPU Speed | CPU to GPU ratio |
|---|---|---|---|---|---|
| 10000 | 12.70 | 13.08 | 0.971 | 0.07 | 0.0057 |
| 20000 | 25.99 | 26.66 | 0.975 | 0.15 | 0.0058 |
| 40000 | 51.02 | 52.34 | 0.975 | 0.26 | 0.0051 |
| 80000 | 96.45 | 99.07 | 0.974 | 0.37 | 0.0038 |
| 160000 | 154.41 | 158.45 | 0.975 | 0.74 | 0.0048 |
| 320000 | 223.43 | 229.10 | 0.975 | 1.43 | 0.0064 |
| 640000 | 280.72 | 287.91 | 0.975 | 2.89 | 0.0103 |
| 1280000 | 330.31 | 338.01 | 0.977 | 5.74 | 0.0174 |
| 2560000 | 393.92 | 402.44 | 0.979 | 10.73 | 0.0272 |
| 5120000 | 517.27 | 526.00 | 0.983 | 22.09 | 0.0427 |
| 10240000 | 530.56 | 530.33 | 1.000 | 45.27 | 0.0853 |
| 20480000 | 511.47 | 512.64 | 0.998 | 78.40 | 0.1533 |
| 40960000 | 479.10 | 481.05 | 0.996 | 127.45 | 0.2660 |
| 81920000 | 476.75 | 478.74 | 0.996 | 185.88 | 0.3899 |
| 163840000 | 476.21 | 478.18 | 0.996 | 231.62 | 0.4864 |
| 327680000 | 475.81 | 477.94 | 0.996 | 246.07 | 0.5172 |
| 655360000 | 475.46 | 477.73 | 0.995 | 263.62 | 0.5545 |

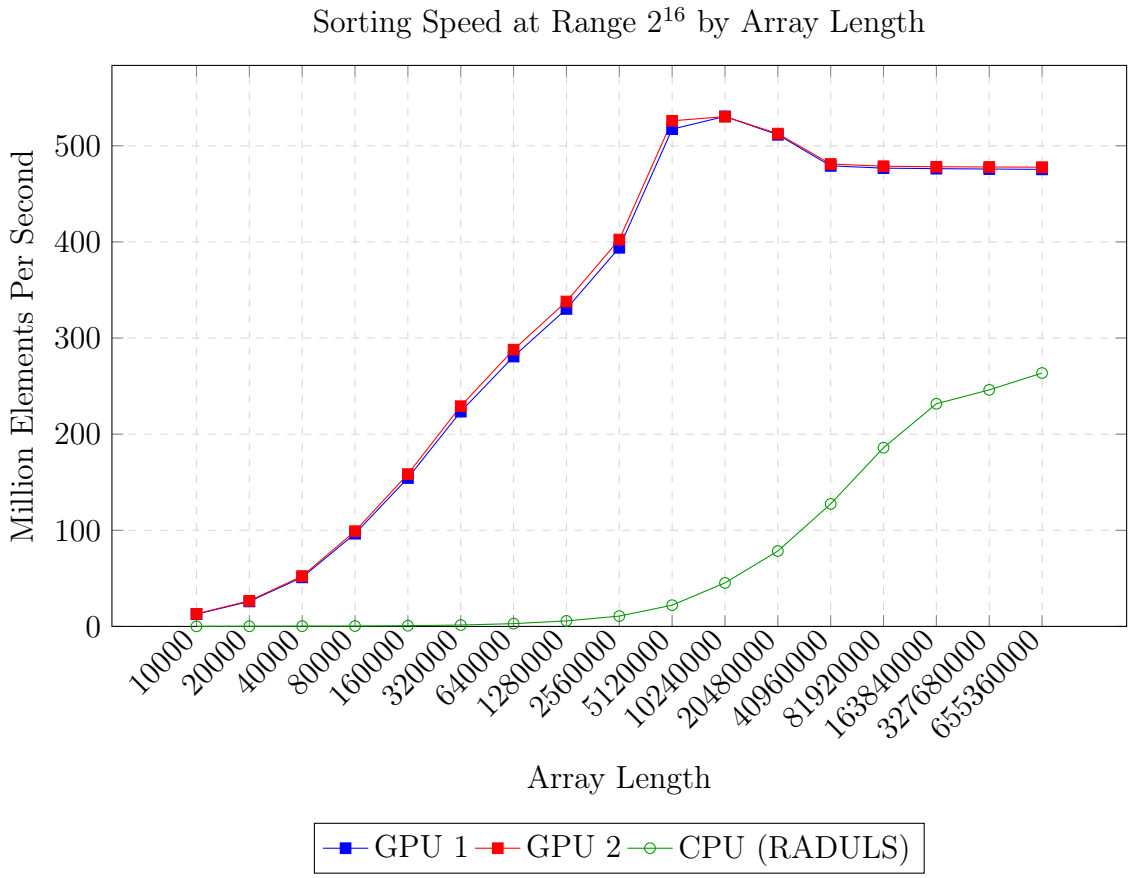


Figure 10: Million Elements Per Second at range 0–$2^{16}$ with varying array length

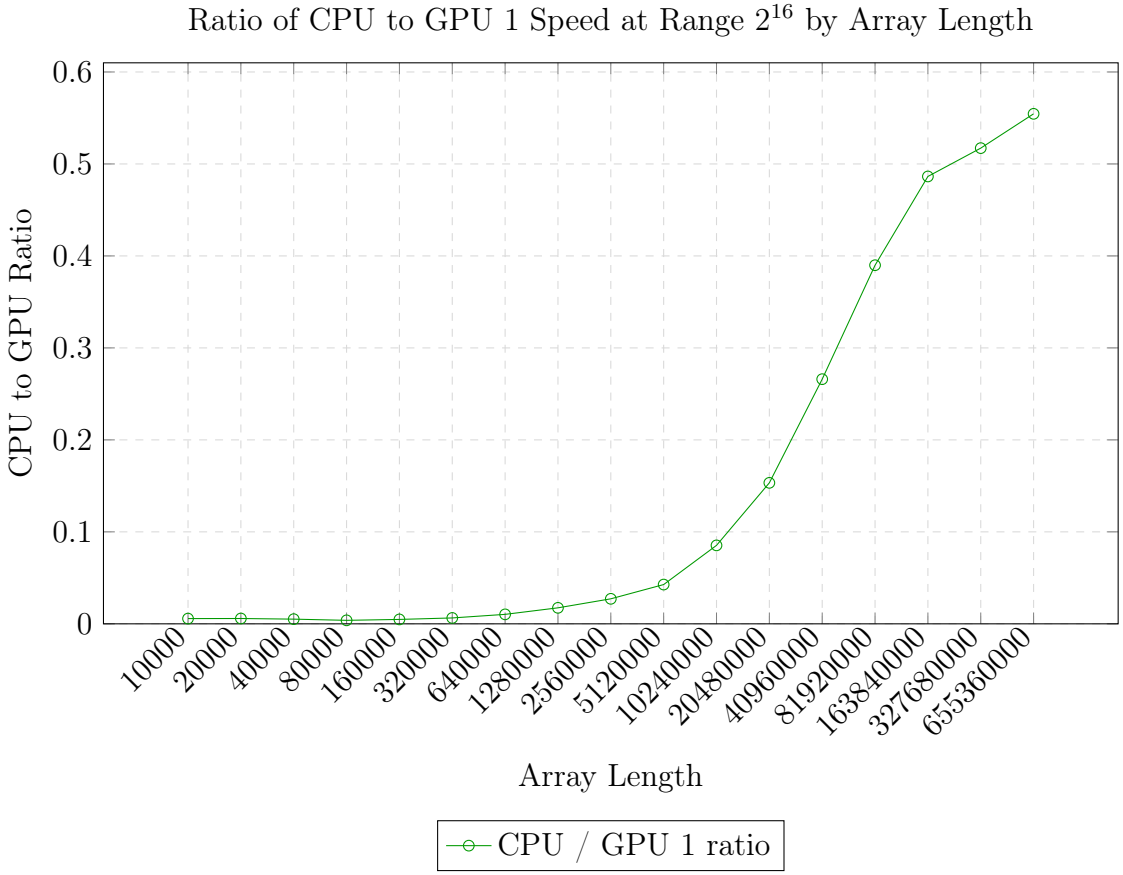


Figure 11: Ratio of CPU to GPU 1 speed at range 0–$2^{16}$ with varying array length. A ratio of 1.0 (red dashed line) indicates equal performance.

The GPU curves behave similarly to the $2^{56}$ case at small sizes, but saturate at a higher peak throughput of approximately 530 MElems/sec, reflecting the reduced number of passes required by Thrust's LSD radix sort for a 2-byte range. The GPU plateau is reached sooner and holds more consistently across large sizes.

The CPU performance, however, is dramatically worse than in the $2^{56}$ case. Despite the smaller range, RADULS throughput at 655 million elements reaches only 263 MElems/sec, compared to 364 MElem/sect at $2^{56}$. The CPU to GPU ratio peaks at just 0.55 and shows no sign of converging towards 1.0. This counterintuitive result, where a narrower range produces worse relative CPU performance, is a consequence of RADULS's MSD archiecture: with value confined to the lower two bytes, the top six bytes of every 64-bit key are zero, causing MSD to descent through six levels of empty buckets before reaching any meaningful distribution.

### 7.2.3 Varying Length Array between ranges $2^{8*1}$ and $2^{8*8}$

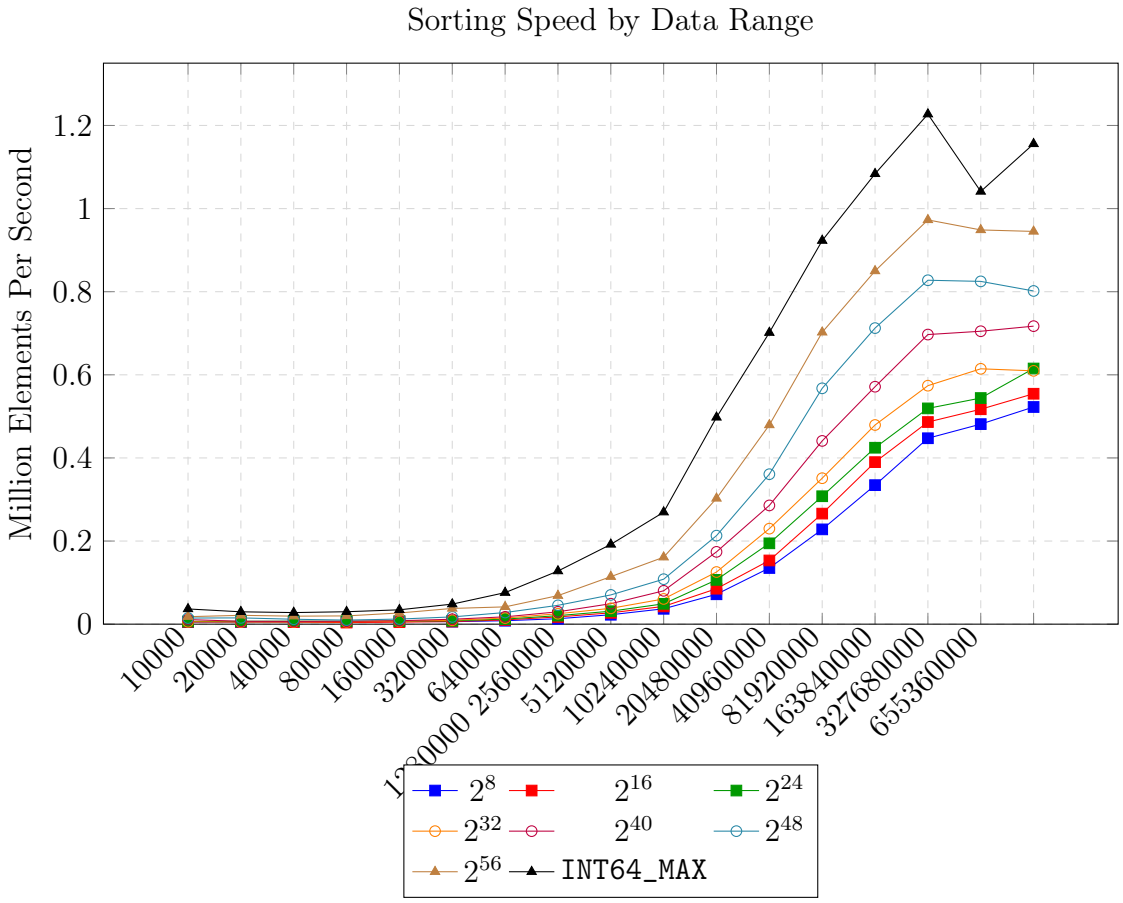


Figure 12: Million Elements Per Second at Array Length 655,360,000

Figure 12 extends the analysis of Section 7.2.1 and 7.2.2 by plotting the CPU to GPU ratio across all eight value ranges simultaneous, from $2^8$ to int64_max, across the full range of array lengths from 10,000 to 655,360,000 elements.

The figure confirms that the two dimensions identifed in earlier sections - array length and value range - interact continuiously and predictably. Every curve shares the same upward trajectory as array length increases, reflecting the GPU's growing parallelism advantage at larger sizes. Simultaneously, high value ranges produce consistently higher ratios at every array length, reflecting the additional LSD passes imposed on the GPU by a wider range. The int_max curve is the only one to cross the ratio of 1.0, reaching 1.23 at 163 million elements, while the $2^8$ curve peaks at just 0.52 and shows no sign of approaching parity.

This has a direct and important implication for the partitioning algorithm. The fundamental performance model requires a speed function $s_i x$ for each processor $i$ as a function of problem size $x$. The data here shows that no single speed function can accurately describe a processor across all workloads - the correct function to use depends on the value range of the input. For a workload with values in the range $0 - 2^{56}$, the CPU speed curve from Table 6 should be used as $s_{CPU}(x)$, and the partitioning algorithm will allocate approximately 49% of elements to the CPU at large size. For a workload with values in the

range $0 - 2^{16}$, the CPU speed curve from Table 7 is the correct input, and the algorithm will allocate only around 35% to the CPU at the same size. Using the wrong speed curve for the given range would cause the algorithm to systematically over or under allocate to the CPU, leaving one or more processors idle while others are still working - the exact inefficiency the algorithm is designed to eliminate.

# 8 Partitioning Algorithm

The most straightforward approach for dealing with this two dimensional variance is to take into account the range of inputs that our array will have before applying the partitioning algorithm. This could be as simple as just looking at the data type being used; in C a char contains 1 byte, an int contains 4 bytes and a long long contains 8 bytes. This means that for each data type, assuming the range will reach up to the maximum value of the data type, a different speed function will apply.

We will start by applying the algorithm to inputs with the maximum range of the long long int data type in C which is 0 - $2^{64}$.

## 8.1 Applying the Algorithm

From Lastovetsky and Reddy 2007

**Algorithm 1.1** Approximation of the $n_i$ so that either $n_i = \lfloor x_i \rfloor$ or $n_i = \lfloor x_i \rfloor - 1$ for $1 \leq i \leq p$ where $\frac{x_1}{s_1(x_1)} = \frac{x_2}{s_2(x_2)} = ... = \frac{x_p}{s_p(x_p)}$ and $x_1 + x_2 + ... + x_p = n$ :

1. The upper line U is drawn through the points $(0, 0)$ and $(\frac{n}{p}, max_i\{s_i\{\frac{n}{p}\}\})$, and the lower line L is drawn through the points $(0, 0)$ and $(\frac{n}{p}, min_i\{s_i\{\frac{n}{p}\}\})$, as shown in Figure 7.

2. Let $x_i^{(U)}$ and $x_i^{(L)}$ be the coordinates of the intersection points of lines U and L with the function $s_i(x)(1 \leq i \leq p)$. If there exists $i \in 1, ..., p$ such that $x_i^{(L)} - x_i^{(U)} \geq 1$ then go to step 3 else go to step 5.

3. Bisect the angle between lines U and L by the line M. Calculate coordinates $x_i^{(M)}$ of the intersection points of the line M with the function $s_i(x)$ for $1 \leq i \leq p$.

4. If $\sum_1^p x_i^{(M)}$ then U = M else L = M. Repeat step 2.

5. Approximate the $n_i$ so that $n_i = \lfloor x_i^{(U)} \rfloor$ for $1 \leq i \leq p$.

## 8.2 Phase 1

The vertical blue line correctly marks $n/p = 524,288,000/3 \approx 1.75 * 10^8$.

The green ray (U) passes through the origin and the point (n/p, thrust speed at (n/p)) - the max speed at n/p which is the GPU at $\sim 3182$.

The red ray (L) passes through the origin and the point (n/p, tbb speed at (n/p)) - the max speed at n/p which is the CPU at $\sim 650$.

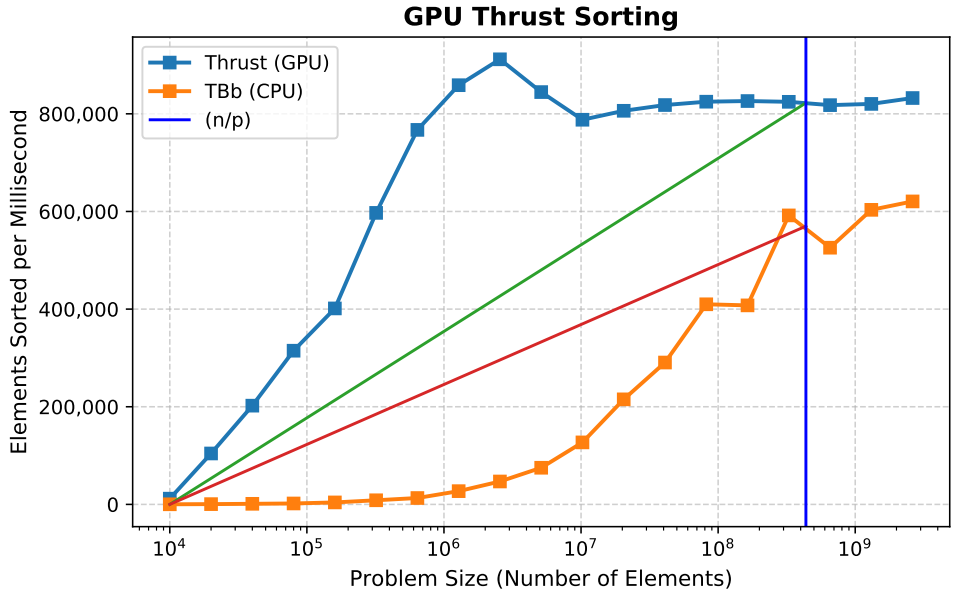


Figure 13:

## 8.3 Phase 2, 3 & 4

This phase of the algorithm effectively performs a binary search on the two slopes, looking for the unique slope where the intersection sums exactly to n. This algorithm runs in a time complexity of O(log n) - just like binary search, it eliminates half the remaining solution space on every iteration.

## 8.4 Phase 5

**Algorithm 1.2** Iterative incrementing of some $n_i$ until $n_1 + n_2 + ... + n_p = n$ :

1. If $n_1 + n_2 + ... + n_p < n$ then go to step 2 else stop the algorithm
2. Find $k \in \{1, ..., p\}$ such that $\frac{n_k+1}{s_k(n_k+1)} = min_{i=1}^{p} = \{\frac{n_i+1}{s_i(n_i+1)}\}$.
3. $n_k = n_k + 1$ Repeat step 1.

Writing this algorithm in pseudocode gives us

**Algorithm 1** Algorithm 1.2

$n$ — total chunks, $n_i$ — initial allocations, processors with speed functions $s_i(x)$ $n_i$ — optimal chunk allocation summing to $n$

**while** true **do** totalSum $\leftarrow \sum_{i=1}^{p} n_i$

  **if** totalSum $\geq n$ **then return** $n_i$  minimum $\leftarrow \infty$  $k \leftarrow 0$

    **for** $i = 1$ $p$ **do** $t \leftarrow \dfrac{n_i + 1}{s_i(n_i + 1)}$

      **if** $t <$ minimum **then** minimum $\leftarrow t$  $k \leftarrow i$  $n_k \leftarrow n_k + 1$

# 9 Application Of the Partitioning Algorithm

In this section, we apply the set partitioning algorithm to a parallel sorting algorithm application using partitioning of an input array on a network of p heterogeneous computers. Our aim in this section is not to show how radix sort algorithms can be efficiently parallelized but to explain in simple terms how the set partitioning algorithm using the functional model can be applied to optimally schedule computational tasks on a network of heterogeneous computers.

## 9.1 Application on Inputs with Wide Element Range

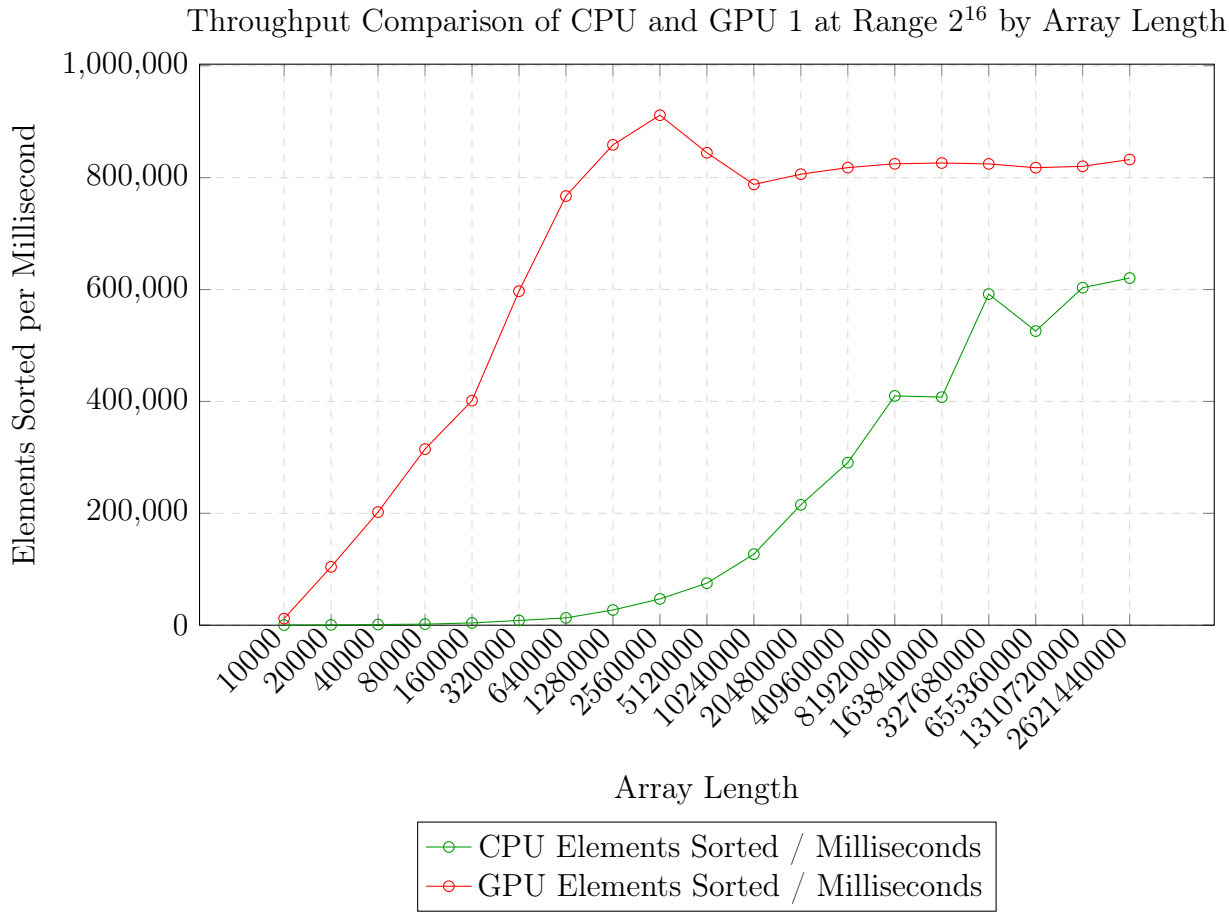


Figure 14: Sorting throughput in elements per millisecond of CPU (RADULS) and GPU 1 (Thrust) across array lengths from 10,000 to 2,621,440,000 elements at value range 0–$2^{56}$.

Spread is calculated by $maxRuntime - minRuntime$.

Balance is calculated by $1 - spread/maxRuntime$.

Table 8: Array size given to each processor when input has range $0 - 2^{56}$ in functional partition

| Array Size | GPU 1 | GPU 2 | CPU |
|---|---|---|---|
| 100000000 | 48415583 | 48415583 | 3168834 |
| 200000000 | 87774916 | 87774916 | 24450169 |
| 300000000 | 126350639 | 126350639 | 47298722 |
| 400000000 | 161258142 | 161258142 | 77483716 |
| 500000000 | 200340776 | 200340776 | 99318448 |

Table 9: Runtime analysis of functional model in (ms) with range $0 - 2^{56}$ in functional partition

| Array Size | GPU 1 (ms) | GPU 2 (ms) | CPU (ms) | Spread (ms) | Balance |
|---|---|---|---|---|---|
| 100000000 | 70 | 65 | 64 | 6 | 91% |
| 200000000 | 130 | 121 | 107 | 23 | 82% |
| 300000000 | 183 | 191 | 170 | 21 | 89% |
| 400000000 | 232 | 241 | 250 | 18 | 92% |
| 500000000 | 286 | 272 | 268 | 18 | 93% |

Table 10: Array size given to each processor when input has range $0 - 2^{56}$ in benchmark test

| Array Size | GPU 1 | GPU 2 | CPU |
|---|---|---|---|
| 100000000 | 33333334 | 33333333 | 33333333 |
| 200000000 | 66666667 | 66666667 | 66666666 |
| 300000000 | 100000000 | 100000000 | 100000000 |
| 400000000 | 133333334 | 133333333 | 133333333 |
| 500000000 | 166666667 | 166666667 | 166666666 |

Table 11: Runtime analysis of benchmark model in (ms) with range $0 - 2^{56}$

| Array Size | GPU 1 (ms) | GPU 2 (ms) | CPU (ms) | Spread (ms) | Balance |
|---|---|---|---|---|---|
| 100000000 | 55 | 44 | 85 | 41 | 51% |
| 200000000 | 101 | 92 | 156 | 64 | 41% |
| 300000000 | 150 | 146 | 218 | 72 | 33% |
| 400000000 | 192 | 189 | 276 | 87 | 68% |
| 500000000 | 240 | 235 | 375 | 140 | 62% |

Figure 14 plots the raw sorting throughput of the CPU and GPU across array lengths from 10,000 to 2,621,440,000 elements at value range $0 - 2^{56}$. The GPU throughput saturates early and remains flat across large sizes, while the CPU throughput grows steadily and

approaches the GPU ceiling at very large array lengths, consistent with the ratio data established in Section 7.2.1.

Table 8 shows the allocations produced by the functional partitioning algorithm for this range. At 100 million elements the CPU receives only 3,168,834 elements — just over 3% of the total — reflecting the large performance gap at small problem sizes where the CPU has not yet reached its throughput plateau. As array size grows, the CPU's allocation increases substantially, reaching 99,318,448 elements at 500 million, approximately 20% of the total. This reflects the algorithm correctly tracking the CPU's improving relative throughput as problem size increases.

Table 9 shows the resulting runtimes. All three processors finish within a narrow window, achieving a balance of between 82% and 93% across the tested array sizes. The spread never exceeds 23ms, demonstrating that the functional model successfully equalizes the finish times of two very different processor types.

For comparison, Table 10 shows the naive equal partition, giving each processor exactly n/3 elements. Table 11 reveals the consequence: the CPU, receiving far more work than it can complete in time, becomes the bottleneck in every run. At 500 million elements the CPU takes 375ms compared to 240ms and 235ms for the two GPUs, producing a spread of 140ms and a balance of only 62%. Across all sizes the equal partition balance ranges from 33% to 68%, a significant degradation compared to the functional model.

## 9.2 Application on Inputs with Small Element Range

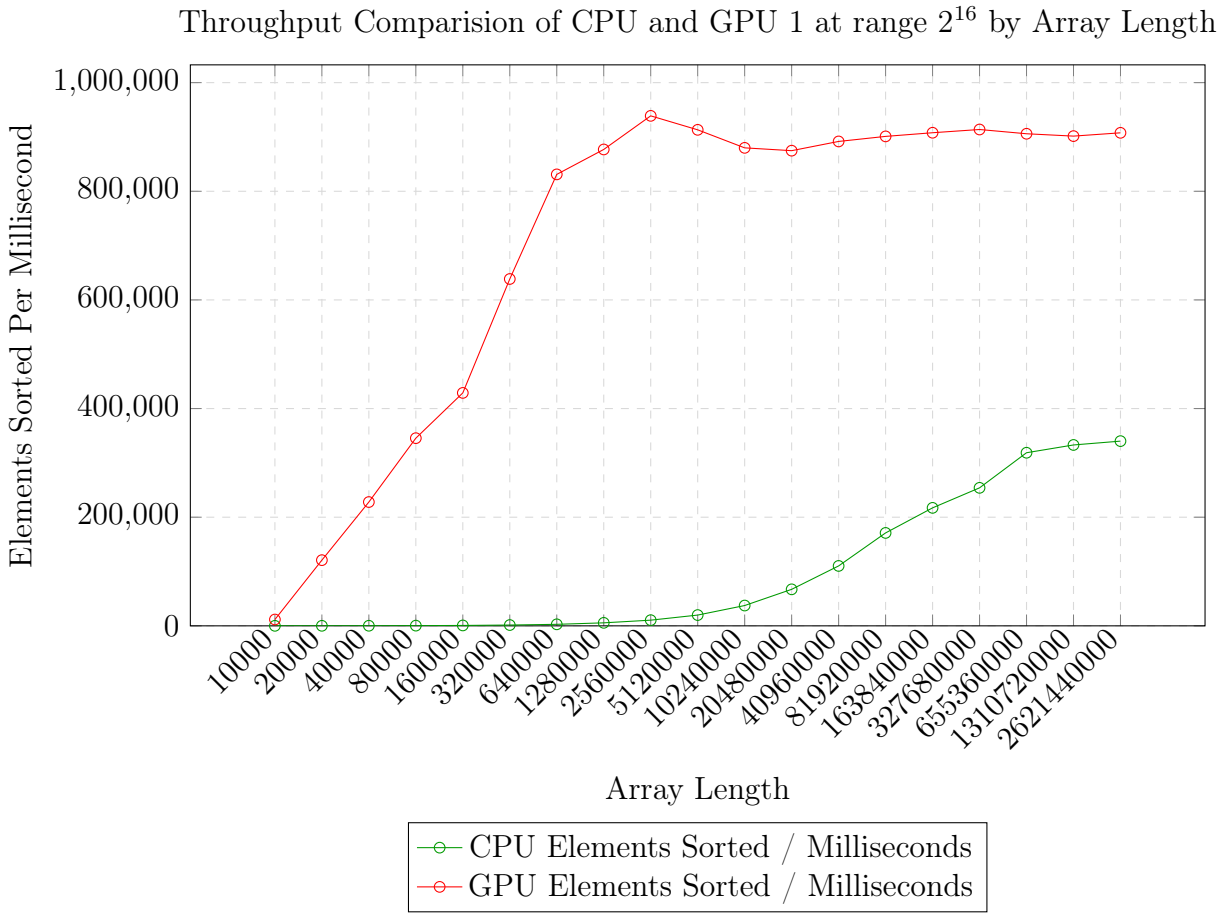


Figure 15: Sorting throughput in elements per millisecond of CPU (RADULS) and GPU 1 (Thrust) across array lengths from 10,000 to 2,621,440,000 elements at value range 0–$2^{16}$.

Table 12: Array size given to each processor when input has range $0 - 2^{16}$ in functional partition

| **Array Size** | **GPU 1** | **GPU 2** | **CPU** |
|---|---|---|---|
| 100000000 | 49995000 | 49995000 | 10000 |
| 200000000 | 99985000 | 99985000 | 30000 |
| 300000000 | 147614722 | 147614721 | 4770557 |
| 400000000 | 196592459 | 196592459 | 6815082 |
| 500000000 | 244592459 | 244592459 | 10815082 |

Table 13: Runtime analysis of functional model in (ms) with range $2^{16}$

| Array Size | GPU 1 (ms) | GPU 2 (ms) | CPU (ms) | Spread (ms) | Balance |
|---|---|---|---|---|---|
| 100000000 | 62 | 58 | 71 | 13 | 81% |
| 200000000 | 118 | 112 | 126 | 23 | 88% |
| 300000000 | 171 | 178 | 159 | 19 | 89% |
| 400000000 | 218 | 224 | 211 | 13 | 93% |
| 500000000 | 268 | 258 | 274 | 18 | 94% |

Table 14: Runtime analysis of benchmark model in (ms) with range $2^{16}$

| Array Size | GPU 1 (ms) | GPU 2 (ms) | CPU (ms) | Spread (ms) | Balance |
|---|---|---|---|---|---|
| 100000000 | 67 | 65 | 318 | 253 | 21% |
| 200000000 | 139 | 138 | 430 | 292 | 32% |
| 300000000 | 210 | 208 | 481 | 271 | 43% |
| 400000000 | 279 | 277 | 576 | 299 | 48% |
| 500000000 | 350 | 347 | 799 | 353 | 50% |

Figure 16 plots throughput value range 0-$2^{16}$. THe GPU curve saturates at a higher peak than in the $2^{56}$ case, consistent with Table 7, while the CPU curve grows much more slowly and reaches a considerably lower ceiling, reflecting the MSD architecture's penalty when values are confined to the lower two bytes of a 64-bit key.

Table 12 shows the functional algorithm's response to this changed performance landscape. At 100 million elements the CPU receives just 10,000 elements, effectively negligible, and even at 500 million elements it receives only 10,812,082, around 2% of the total. The algorithm has correctly identified that the CPU is far less competitive at this range and has reduced its allocation accordingly. The two GPUs absorb nearly all the work symmetrically, consistent with their near identical speed curves from Table 7.

Table 14 shows that the equal partition benchmark performs considerably worse at this range than at $2^{56}$. With each processor receiving n/3 elements, the CPU is severely overloaded relative to its actual throughput at $2^{16}$. Balance falls to just 21% at 100 million elements and does not exceed 50% across any tested size, confirming that equal partitioning is a poor strategy when the CPU's relative weakness is as pronounced as it is at narrow value ranges.

## 9.3 Difference Between Them

The most striking difference between the two cases is the CPU allocation produced by the partitioning algorithm. At range 0-$2^{56}$ the CPU receives between 3% and 20% of elements depending on array size, a meaningful and growing share the reflects the CPU's near parity throughput at large sizes seen in Table 6. At range 0-$2^{16}$ the CPU receives under 3% at every tested size, and at small arrays it receives almost nothing at all. In both cases the functional model achieves good balance - but it does so by applying fundamentally different allocations depending on the range.

This illustrates directly why the selection of range-based speed functions discussed in Section 7.2.3 is essential: providing the $2^{56}$ speed curve for a $2^{16}$ workload would cause the algorithm to assign the CPU roughtly 20% rather than 2%, leaving the CPU still sorting long after the GPU has finished.

This also shows clearly that the effectiveness of a sorting algorithm depends on many things, not just the length of the input, but also its respective best and worst cases, this raises the idea that a fully production ready heterogeneous sorting system would make use of multiple sorting algorithms depending on the input.

### 9.4 Merging Step

Once each processor has sorted its partition, the results must be combined into a single fully sorted array. Since each partition is independently sorted but the values across the partition may interleave, a two way merge is required. This is performed on the CPU using a linear merge operation O(n) which is negligible compared to the O(n*d) cost of the radix sort itself. For p processor case used in this paper a simple sequential merge is sufficient. Importantly, the merge cost was not included in the runtime measurements of Tables 8 and 9, which represent a minor limitation - at 500 million elements the merge adds approximately 1-2 seconds on top of the sort, and including it would slightly reduce the balance figures reported.

## 10 Using Multiple Sorting Algorithms

It is important to take into consideration that sorting is quite a different problem to something like matrix multiplication which the partitioning algorithm is based on because there are dozens of sorting algorithms each with their own benefits and downsides. I chose to use Radix sort because it is the most efficient algorithm for large datasets and is not dependent on the ordering of the inputted data, however for small datasets or datasets which are already mostly ordered it will have significantly worse than Merge or Quick sort, a mature program for sorting on a heterogeneous system should not be limited to one sorting algorithm and will take these issues into account.

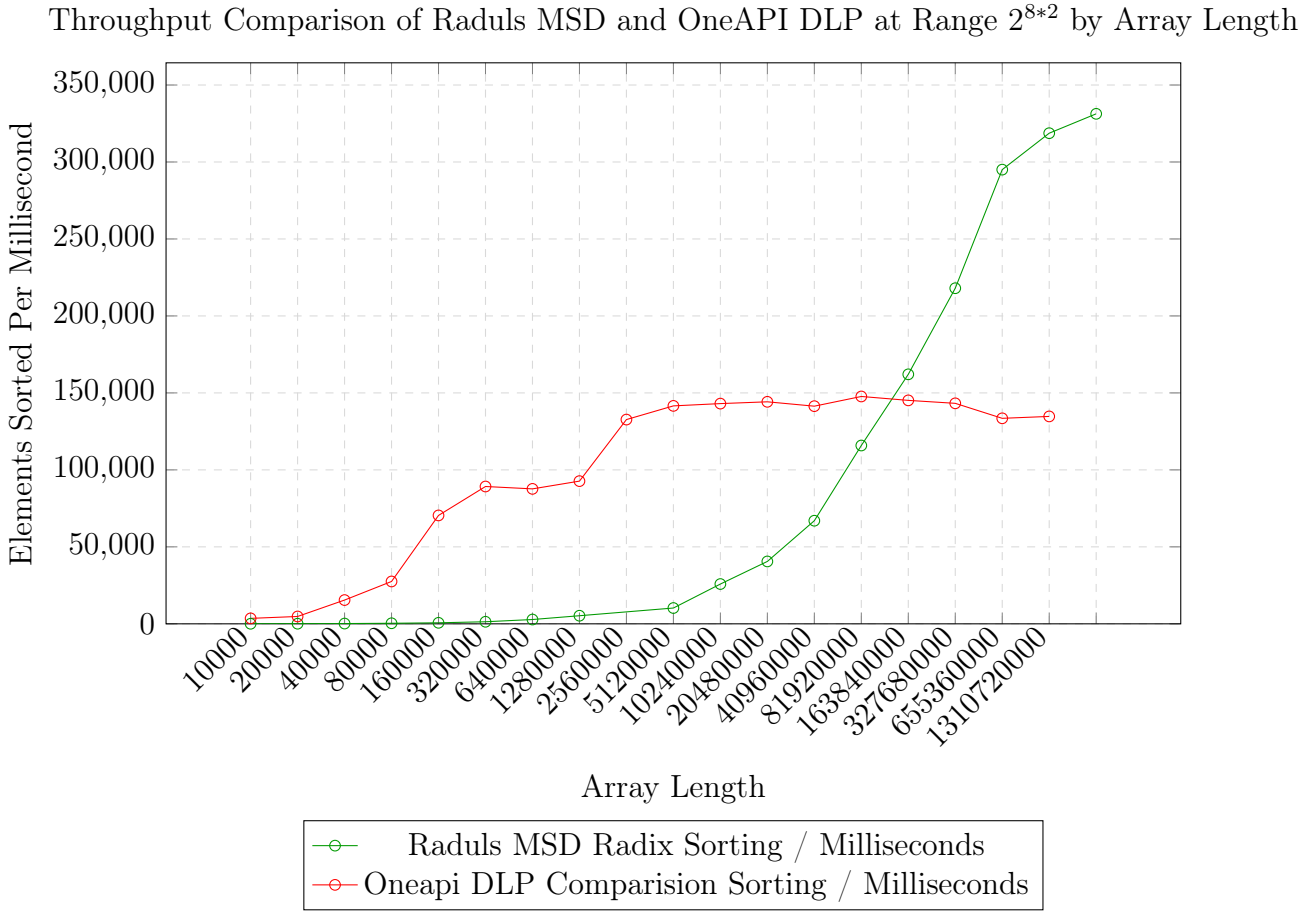


Figure 16: Sorting throughput in elements per millisecond of Raduls MSD Radix Sort and OneAPI DLP comparison sort across array lengths from 10,000 to 1,310,720,000 elements at value range 0–$2^{2*2}$.

However, knowing when it is optimal to switch between different sorting algorithms is itself a non trivial problem because of how best and worst cases are different depending on the sorting algorithm. In the graph above we can see a single point around 100,000,000 elements where the Radix sort overtakes the comparison based sorting algorithm. However a simple change to the range of inputs from 0-$2^{8*2}$ to 0-$2^{8*7}$ changes where the point is dramatically.

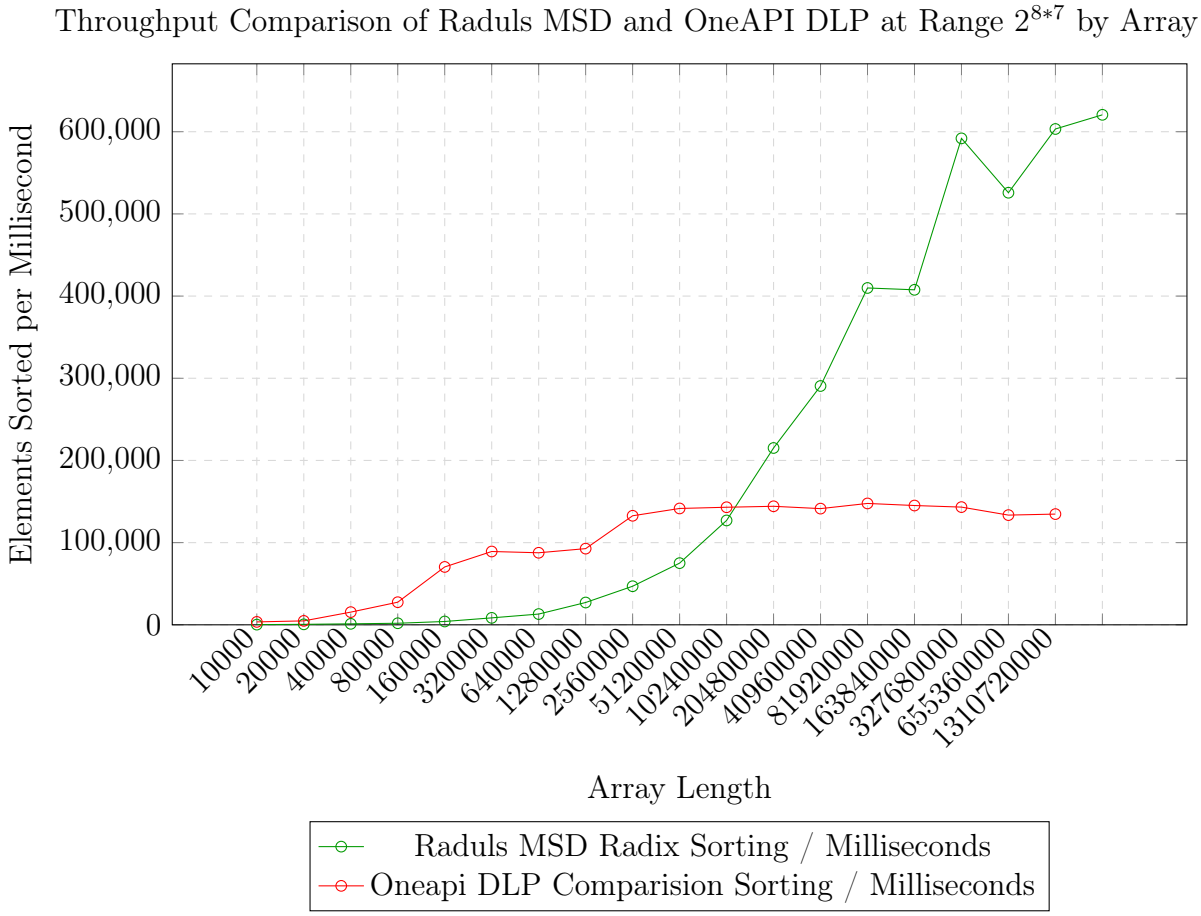


Figure 17: Sorting throughput in elements per millisecond of Raduls MSD Radix Sort and OneAPI DLP comparison sort across array lengths from 10,000 to 1,310,720,000 elements at value range 0–$2^{2*7}$.

Now with a input array which favors MSD Radix Sort the point at which MSD Radix Sort overtakes TBB's comparison based sorting algorithm is moved from 100,000,000 to 10,000,000 elements.

# 11 Things To Take Into Consideration

Because heterogeneous servers are typically shared between multiple people it is often true that a partitioning algorithm might not be applicable if someone else is making use of their compute ability or memory. With the rising cost of hardware and development in networking ability shared resources will become more prevalent, it is important to coordinate any programs on your shared server with other users of your server.

This is especially important for the algorithms described in this paper which try to create an equal runtime between all the processors in our systems. There were many times when I would get very unexpected results which confused me, but after I checked using system monitoring tools such as $ htop I realized that simply other students were using the system at the same time.

# 12 Conclusion

In this paper, we investigated the problem of optimally distributing a sorting workload across the heterogeneous processors of a hybrid server featuring one Intel Icelake CPU and two NVIDIA A40 GPUs. We began by establishing that radix sort is the most appropriate algorithm for this use case, demonstrating that unlike comparison based algorithms, its runtime is independent of the ordering of the input array, making it well suited for a partitioning algorithm that must produce consistent allocations regardless of input.

We then characterized the performance of two parallel radix sort implementations, NVIDIA Thrust's LSD radix sort on the GPU and the Raduls MSD radix sort on the CPU - and found that their throughput varies along two distinct dimensions; array length and value range. Crucially, these two algorithms respond to increasing value range in opposite directions, with GPU performance degrading as range widens due to additional LSD passes, while CPU performance improves due to the early termination properties of MSD Recursion. This two dimensional variance means that no single speed function can accurately describe either processor across all workloads, and the correct speed function to supply to the partitioning algorithm depends on the value range of the input data.

Applying the functional performance model of Lastovetsky and Reddy to our measured speed functions, we demonstrated that the partitioning algorithm achieves a workload balance of up to 94% across all three processors, compared to just $28\% - 35\%$ for a naive equal partition benchmark. This represents a significant improvement in hardware utilization and confirms that the functional performance model, despite being originally designed for matrix multiplication on homogeneous networks, can be successfully applied to sorting on a heterogeneous server when the speed functions are correctly calibrated.

Several directions remain open for future work. Extending the performance model to treat both array length and value range as simultaneous inputs to the speed function would allow the partitioning algorithm to operate optimally across all workloads without requiring prior knowledge of the data range. Additionally, incorporating a switching mechanism between radix sort and comparison based algorithms for small or nearly sorted inputs, and accounting for the merge step overhead in the balance measurements, would bring the implementation close to a production ready heterogeneous sorting system.

# References


[1] NVIDIA Corporation, *NVIDIA A40 GPU Accelerator Product Brief*, PB-09976-001_v08, April 2022.

[2] S. Farrelly, R. R. Manumachu and A. Lastovetsky, *OpenH: A Novel Programming Model and API for Developing Portable Parallel Programs on Heterogeneous Hybrid Servers*, 2024.

[3] Rohit Chandra, *Parallel Programming in OpenMP*, 2000.

[4] Dick Buttlar, Jacqueline Farrell, and Bradford Nichols, *Pthreads Programming: A POSIX Standard for Better Multiprocessing*, O'Reilly Media, Inc., 1996.

[5] T. Hoshino, N. Maruyama, S. Matsuoka and R. Takaki, *CUDA vs OpenACC: Performance Case Studies with Kernel Benchmarks and a Memory-Bound CFD Application.*

[6] NVIDIA Corporation, *CUDA Compiler Driver NVCC*.

[7] Wen-mei W. Hwu, Izzat El Hajj, *Programming Massively Parallel Processors*, Chapter 13.

[8] Chuck Pheatt, *Intel® Threading Building Blocks*, Journal of Computing Sciences in Colleges, 23(4):298–298, 2008.

[9] Alexey Lastovetsky and Ravi Reddy, *Data Partitioning with a Functional Performance Model of Heterogeneous Processors*, The International Journal of High Performance Computing Applications, 21(1):76–90, 2007.

[10] Intel Corporation, *Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference*, Intel Corporation, 2024.

[11] Richard Stallman and the GCC Developer Community, *Using the GNU Compiler Collection (GCC)*, Free Software Foundation, 2024.